

Work is multiplayer. Agents are single-player.

- Every agent you use today lives in exactly one person's window.
- So the human becomes the integration layer: copy-paste between two chat windows.
- A screenshot forwarded to the other side. “Let me ask the AI and get back to you.”
- The second person never sees the context.
- The deliverable dies inside one private thread.

HOW IT WORKS — AND HOW YOU GET IN

Text a bot. Share a code. That's the whole onboarding.

MEMBER	SURFACE	SENDS	RECEIVES	JOINED BY
Julie	phone — Telegram	the budget, the hard constraints	terse replies, voice, files	answering a message
Tom	phone — Telegram	voice notes, the venue photo	terse replies, voice, files	answering a message
Atlas	local desktop agent — a machine	the commit log, the files, what the laptop knows	the same transcript as the humans	running one script
the boss	email	reply-to-thread	the PDF, once a member confirms	replying once

You text “new” to a bot and the room exists. You send anyone the code or the QR, and they are in — from a phone, a browser, or by replying to an email. No install, no shared login, no new app. And because the room models members rather than humans, a machine joins by running one script and is treated exactly like a person.

ILLUSTRATION

Telegram thread: “new” → the room code + QR reply, then a second phone joining with “join RDV-7F3K”. Shot from two real phones side by side.

IT WORKS — LIVE, ON REAL SURFACES

Not a mock. A real phone, a real thread, a real artifact.

- 0s

A real phone types “new” on Telegram — the room exists
- 86.5s

Cold boot: agent session and shared artifact both ready
- attributed

A laptop joins; one agent reply reaches both surfaces, each message tagged with who sent it and from where
- live edit

The agent edits the shared artifact on request — confirmed on the URL while the phone thread continues
- delivered

A PDF previewed, confirmed with a token, rendered — and byte-matched in the inbox
- survived

The sandbox deleted out from under the room; the sweep caught it and the next message revived it
- shown in the video —

voice in · photo in · a private “@me” answer · a voice reply · real files attached · a desktop agent joining as a member · resume replaying the prior transcript

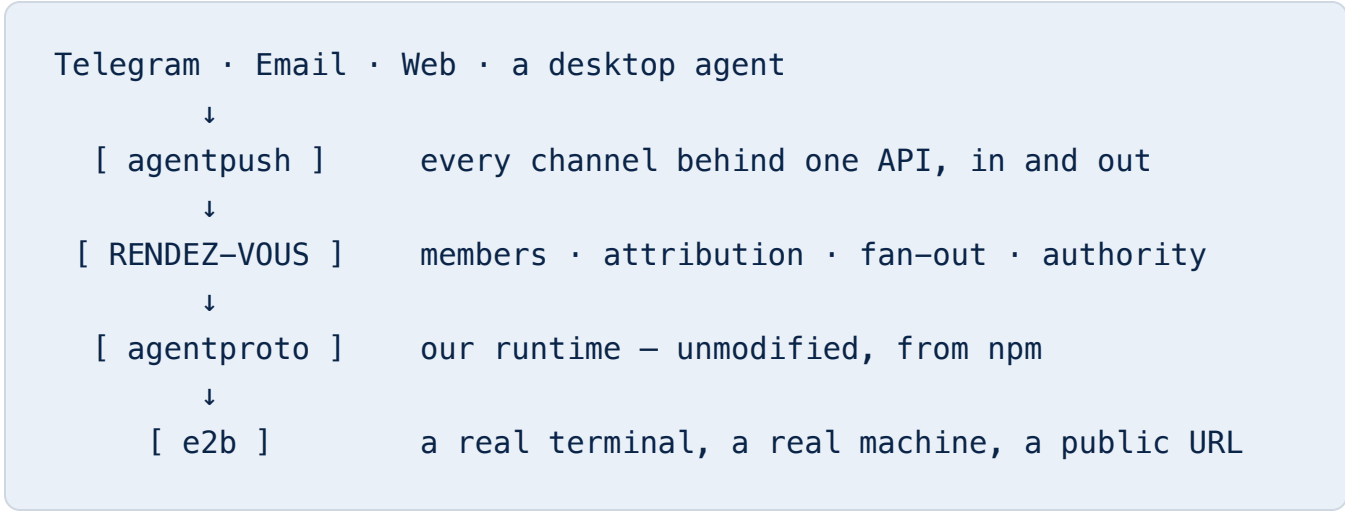
The rows above the line are in the rehearsal log with timestamps. The row below it is built and tested, and you watch it happen — real screens, one take. We keep the two apart on purpose.

ILLUSTRATION

The 4-up demo frame: Julie's phone, Tom's phone, the room web view with transcript + live artifact, and the Atlas terminal strip. One clock across the top.

A shared room on top of a real machine.

- agentpush — every channel behind one API, in and out. The room never learns a provider's quirks.
- Rendez-vous — the room: members, attribution, fan-out by presence tier, the authority to send.
- agentproto — our own open-source agent runtime. Unmodified, straight from npm.
- e2b — one sandbox per room: nobody's laptop, an isolated environment everyone can trust.



Not an API behind a chat box — a full harness. A real terminal, a real machine, and a public URL: when the agent builds a PDF or a site, everyone sees the same thing at the same address, and can change it.

ILLUSTRATION

The room web page on a laptop: attributed transcript on the left, the artifact updating live on the right — plus the served artifact URL in the browser bar.

A room with a notion of authority, not a relay.

- 1 A member asks the room for the deliverable — a PDF of what everyone just built
- 2 **THE CONFIRMATION GATE**
the room previews it first: recipient, channel, subject, the rendered document — and sends nothing until a member explicitly confirms
- 3 it lands on a phone as a real PDF and in an inbox — recipient, subject and contents exactly as previewed
- 4 the send is written into the shared transcript: who asked, who confirmed, what, to whom

Every other capability keeps work inside the room. This one lets it leave — under the members' control, on the record.

ILLUSTRATION

The Ternwood PDF and the live employee site side by side, both rendered from the same data.json through the same design kit — and the phone showing the confirm gate.

The room is the primitive.

The runtime is a detail.

Eight findings, and where each fix landed.

- 1

PR 1274 · PR 1273

a mid-turn prompt is lost without queue: true, and a killed session still answers 200 so resume reports success doing nothing
- 2

PR 1279 · PR 1278 · PR 1286

sandbox liveness unreported; orphaned boxes never reaped; a failed reconnect leaves the box running and billing
- 3

PR 1281 · PR 1277

app serve ignores the declared UI path; a mutating proxy route had no auth gate
- 4

FIXED IN THE ROOM

the fan-out cursor froze on resume, the artifact link outlived its box, and a recap fix shipped dead on arrival — each now has a regression test
- 5

DOCUMENTED

an account-pinned inbound route never fires; the agent proposes rebuilding a capability the workspace already brokered

The recap fix is the one worth telling: its first version never ran, hid behind a reply that looked exactly like success, and 443 passing tests never caught it. A local harness did.

This is why the room records who asked, who confirmed, and what was sent. Where failure is silent, the transcript is the only thing checkable afterwards.

What we have not proven.

- Paused sandboxes expire in 20–60 minutes. One did, live, mid-rehearsal — the room advertised a dead link until we made box liveness its own fact.
- WhatsApp is not provisioned: no number, no key. The room is channel-agnostic by construction, so it is a config change, not code.
- We share an existing workspace's Telegram routing; a catch-all route there could fire alongside ours.
- Swapping the runtime for Codex failed at the auth gate, not the swap — a fresh box has no login. The credential model is the real work.